МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХМЕЛЬНИЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
Факультет інформаційних технологій
Кафедра комп'ютерної інженерії та інформаційних систем

Лабораторна робота №1

з дисципліни «Кросплатформне програмування»

на тему: «Базові операції у Java: змінні, методи та класи»

Виконав:
студент 2 курсу, групи ІСТ-25-1 _____ Хатько О.Г.
(Підпис)

Перевірив:
ст. викл. _____ Денисюк Д.О.
(Підпис)

Хмельницький – 2026

Лабораторна робота №1

Мета. Налаштувати середовище розроблення Java-проєктів і реалізувати консольну програму, у якій обчислення зосереджено в предметному класі. Результат має відтворюватися засобами Maven, проходити модульні тести та зберігатися у загальнодоступному репозиторії Git.

Результати навчання

Після виконання роботи студент повинен:

1. **Знати** призначення JDK, компілятора `javac`, байткоду та JVM;
2. **Знати** роль Maven, JUnit і Git у відтворюваному процесі розроблення та перевірки;
3. **Вміти** створювати й відкривати Maven-проєкт в Apache NetBeans та виконувати його складання;
4. **Вміти** оголошувати змінні, обирати типи й реалізовувати методи з параметрами та значеннями, що повертаються;
5. **Вміти** описувати клас із полями, конструктором і методами, а також створювати об'єкти оператором `new`;
6. **Вміти** реалізовувати модульні тести, виконувати перевірку з чистої папки та публікувати зафіксовану версію проєкту в Git.

№6 варіант

Виконання роботи

1. Таблиця вхідних даних, типів, одиниць вимірювання та допустимих діапазонів

Параметр (Поле)	**Тип даних**	**Одиниця вимірювання**	**Допустимий діапазон / Умова**	**Опис**
previous	double	кВт·год	≥ 0, скінченне число	Попередній показник лічильника
current	double	кВт·год	≥ previous, скінченне число	Поточний показник лічильника
tariff	double	грн / кВт·год	> 0, скінченне число	Тариф за одиницю споживання
days	int	дні	> 0, (ціле число)	Кількість днів розрахункового періоду

2. Математичні формули варіанта і пояснення порядку обчислень

   Обсяг споживання електроенергії (consumption):

   Consumption = current - previous

   Пояснення: Визначає загальну кількість спожитої електроенергії за розрахунковий період як різницю між поточним та попереднім показниками лічильника.

   Загальна вартість (cost):

   Cost = Consumption * tariff

Пояснення: Розраховується шляхом множення загального обсягу спожитої електроенергії на встановлений тариф.

Середньодобове споживання (averageDailyConsumption):

AverageDaily = Consumption / days

Пояснення: Визначає інтенсивність споживання електроенергії на день шляхом ділення загального обсягу споживання на кількість днів у розрахунковому періоді.

3. Структура Maven-проєкту та призначення основних файлів

- pom.xml — головний декларативний файл конфігурації Maven (Project Object Model); містить інформацію про координати проєкту, версію Java (реліз 21), підключені плагіни складання та залежності (зокрема JUnit).

- submission.properties — службовий файл конфігурації для автоматизованої системи перевірки; містить лише номери лабораторної та варіанта (lab=1, variant=6).

- src/main/java/ua/khnu/kpp/lab1/ElectricityUsage.java — предметний клас, що містить бізнес-логіку, поля, захищений конструктор та обчислювальні методи.

- src/main/java/ua/khnu/kpp/lab1/Main.java — консольна точка входу (main), яка зчитує токени через Scanner, ініціалізує об'єкт та формує стандартизований вивід (RESULT_N або ERROR).

- src/test/java/ua/khnu/kpp/lab1/ElectricityUsageTest.java — власний клас модульних тестів студента для перевірки контрольних значень і валідації.

- src/test/java/ua/khnu/kpp/lab1/Lab1PublicTest.java — фіксовані публічні тести викладача, які перевіряють дотримання публічного контракту.

- .gitignore — файл конфігурації Git, що визначає файли та каталоги, які не повинні потрапляти до репозиторію (наприклад, каталог збірки target/).

4. Таблиця контрольного, альтернативного й негативного тестів

№	Категорія тесту	Вхідні дані (Аргументи)	Очікуваний результат	Фактичний результат	Статус
1	Контрольний тест	12450.0, 12638.0, 4.32, 30	Споживання: 188.0; Вартість: 812.16; Середньодобове: 6.2666...	Споживання: 188.0; Вартість: 812.16; Середньодобове: 6.2666...	**Успішно (Passed)**
2	Альтернативний тест	1000.0, 1250.0, 5.0, 25	Споживання: 250.0; Вартість: 1250.0; Середньодобове: 10.0	Споживання: 250.0; Вартість: 1250.0; Середньодобове: 10.0	**Успішно (Passed)**
3	Негативний тест 1	-5.0, 100.0, 4.32, 30	Викидання винятку IllegalArgumentException	Викинуто IllegalArgumentException	**Успішно (Passed)**

4	Негативний тест 2	200.0, 100.0, 4.32, 30 (current < previous)	Викидання винятку IllegalArgumentException	Викинуто IllegalArgumentException	**Успішно (Passed)**
5	Негативний тест 3	100.0, 200.0, 0.0, 30 (tariff = 0)	Викидання винятку IllegalArgumentException	Викинуто IllegalArgumentException	**Успішно (Passed)**
6	Негативний тест 4	100.0, 200.0, 4.32, 0 (days = 0)	Викидання винятку IllegalArgumentException	Викинуто IllegalArgumentException	**Успішно (Passed)**

Код класів:

```
package ua.khnu.kpp.lab1;

public final class ElectricityUsage {
    private final double previous;
    private final double current;
    private final double tariff;
    private final int days;

    public ElectricityUsage(double previous, double current, double tariff, int
days) {
        if (!Double.isFinite(previous) || previous < 0
                || !Double.isFinite(current) || current < previous
                || !Double.isFinite(tariff) || tariff <= 0
                || days <= 0) {
            throw new IllegalArgumentException("Invalid electricity usage
parameters");
        }
        this.previous = previous;
        this.current = current;
        this.tariff = tariff;
        this.days = days;
    }

    public double consumption() {
        return current - previous;
    }

    public double cost() {
        return consumption() * tariff;
    }

    public double averageDailyConsumption() {
        return consumption() / days;
    }
}
```

```
package ua.khnu.kpp.lab1;

import java.util.Locale;
```

```java
import java.util.Scanner;

public final class Main {
    private static final String ERROR = "ERROR";

    private Main() {
    }

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in).useLocale(Locale.ROOT);

        if (!input.hasNextDouble()) {
            System.out.println(ERROR);
            return;
        }
        double previous = input.nextDouble();

        if (!input.hasNextDouble()) {
            System.out.println(ERROR);
            return;
        }
        double current = input.nextDouble();

        if (!input.hasNextDouble()) {
            System.out.println(ERROR);
            return;
        }
        double tariff = input.nextDouble();

        if (!input.hasNextInt()) {
            System.out.println(ERROR);
            return;
        }
        int days = input.nextInt();

        if (input.hasNext()) {
            System.out.println(ERROR);
            return;
        }

        try {
            ElectricityUsage usage = new ElectricityUsage(previous, current,
tariff, days);
            System.out.printf(Locale.ROOT, "RESULT_1=%.2f%n", usage.consumption());
            System.out.printf(Locale.ROOT, "RESULT_2=%.2f%n", usage.cost());
            System.out.printf(Locale.ROOT, "RESULT_3=%.2f%n",
usage.averageDailyConsumption());
        } catch (IllegalArgumentException exception) {
            System.out.println(ERROR);
        }
    }
}
```

```java
package ua.khnu.kpp.lab1;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;
import org.junit.jupiter.api.Test;

class ElectricityUsageTest {
    private static final double EPSILON = 1e-9;

    @Test
    void calculatesControlValues() {
        ElectricityUsage usage = new ElectricityUsage(12450.0, 12638.0, 4.32, 30);
        assertEquals(188.0, usage.consumption(), EPSILON);
        assertEquals(812.16, usage.cost(), EPSILON);
        assertEquals(188.0 / 30.0, usage.averageDailyConsumption(), EPSILON);
    }

    @Test
    void rejectsInvalidParameters() {
        assertThrows(IllegalArgumentException.class, () -> new ElectricityUsage(-
5.0, 100.0, 4.32, 30));
        assertThrows(IllegalArgumentException.class, () -> new
ElectricityUsage(200.0, 100.0, 4.32, 30));
        assertThrows(IllegalArgumentException.class, () -> new
ElectricityUsage(100.0, 200.0, 0.0, 30));
        assertThrows(IllegalArgumentException.class, () -> new
ElectricityUsage(100.0, 200.0, 4.32, 0));
    }
}
```

Висновок. У ході виконання налаштована середовище розроблення Java-проєктів і реалізовано консольну програму, у якій обчислення зосереджено в предметному класі. Результат відтворюється засобами Maven, проходить модульні тести та зберігається у загальнодоступному репозиторії Git.